

Part A (Zumrud)

- CLIENT CODE (tests.rkt)
- ENVIRONMENT (environments.rkt)

ρ ranges over environments.

$[]$ denotes the empty environment.

$[\text{var} = \text{val}] \rho$ denotes $(\text{extend-env var val } \rho)$.

$[\text{var}_1 = \text{val}_1, \text{var}_2 = \text{val}_2] \rho$ abbreviates $[\text{var}_1 = \text{val}_1](\text{extend-env var}_2 \text{val}_2 \rho)$, etc.

$[\text{var}_1 = \text{val}_1, \text{var}_2 = \text{val}_2, \dots] \rho$ denotes the environment in which the value of var_1 is val_1 , etc.

- INTERFACE (lang.rkt)

```
Program ::= Expression
  a-program (exp1)
Expression ::= Number
  const-exp (num)
Expression ::= (Number / Number)
  rational-exp (num1, num2)
Expression ::= op(Expression, Expression, Number)
  op-exp (exp1, exp2, num)
Expression ::= create-new-list ()
  list-exp ()
Expression ::= cons Expression to Expression
  cons-exp (exp1 lst)
Expression ::= multiplication(Expression)
  mul-exp (lst)
Expression ::= min(Expression)
  min-exp (lst)
Expression ::= zero?(Expression)
  zero?-exp (exp1)
Expression ::= if Expression then Expression
  elif Expression then Expression else Expression
  if-elif-exp (exp1 exp2 exp3 exp4 exp5)
Expression ::= Identifier
  var-exp (var)
Expression ::= let Identifier = Expression in Expression
  let-exp (var exp1 body)
Expression ::= simpl(Expression)
  simpl-exp (exp1)
```

- REPRESENTATION (data-structures.rkt)

```
num-val:      Integer->ExpVal
rational-val: (Integer . Integer)->ExpVal
bool-val:     Boolean->ExpVal
list-val:     List->ExpVal
expval->num:   ExpVal->Integer
expval->rational: Expval->(Integer . Integer)
expval->bool:   ExpVal->Boolean
expval->list:   ExpVal->List
```

- INTERPRETER (interp.rkt)

```
(value-of-program exp) = (value-of exp p)
const-exp:      Integer->Exp
rational-exp:   Integer,Integer->Exp
op-exp:         Exp,Exp,Integer->Exp
list-exp:       Exp
cons-exp:       Exp,Exp->Exp
mul-exp:        Exp->Exp
min-exp:        Exp->Exp
zero?-exp:      Exp->Exp
if-elif-exp:    Exp,Exp,Exp,Exp,Exp->Exp
var-exp:        Var->Exp
let-exp:        Var,Exp,Exp->Exp
simpl-exp:      Exp->Exp
```

Part B (Göktürk)

- ```
(define init-env
 (lambda ()
 (extend-env
 'v (num-val 3)
 (extend-env 'x (num-val 4)
 (extend-env 'y (num-val 2)
 (extend-env 'z (num-val 3)
 (empty-env)))))))
```
- $\rho_0 = []$   
 $[z = [3]] \rho_0 \rightarrow \rho_1 = [z = 3]$   
 $[y = [2]] \rho_1 \rightarrow \rho_2 = [y = 2, z = 3]$   
 $[x = [4]] \rho_2 \rightarrow \rho_3 = [x = 4, y = 2, z = 3]$   
 $[v = [3]] \rho_3 \rightarrow \rho_4 = [v = 3, x = 4, y = 2, z = 3]$
- Environments can be represented as data structures, lambda expressions, and using “define-datatype”. This environment was implemented as a lambda expression.

## Part C (Gul)

Expressed values are associated with return values of expressions while denoted values are associated with variables. Since in this language, variables can only be assigned to an expression, expressed and denoted values are the same:

ExpVal: Integer, Rational number, Boolean, List

DenVal: Integer, Rational number, Boolean, List

## Part D (Göktürk, Zumrud, Gul)

The answers are inside `interp.rkt`

`lang.rkt` – Göktürk, Gul

`environments.rkt` – Göktürk

`data_structures.rkt` – Zumrud, Gul

`interp.rkt` – Zumrud, Göktürk